

Programozás II.

Minta ZH 2. – Labor

A feladat balatoni hajójáratok adatainak feldolgozása és kezelése egy program segítségével. A `jaratok.csv` fájl tartalmazza a járatok adatait, minden mező pontosvesszővel van elválasztva, és minden adatsor új sorba kerül. Az első sor fejléceket tartalmaz. A fájl legfeljebb 200 adatsort tartalmaz!

Készítsen egy **Jarat** absztrakt osztályt, amely a példányok tárolására fog szolgálni. Az osztály minden adattagja legyen bárholonnan hozzáférhető. Az adattagok nevei a következők legyenek:

- `Datum` – karakterlánc,
- `Indulas` – karakterlánc,
- `Erkezes` – karakterlánc,
- `Menetido` – karakterlánc,
- `HajoNeve` – karakterlánc,
- `InduloKikoto` – karakterlánc,
- `CelKikoto` – karakterlánc,
- `Fordald` – egész.

Ezen felül legyen egy paraméteres konstruktora, amely az adattagokat veszi át, és átadja a megfelelő változóknak! Készítsen még egy **Utvonal** metódust, amely visszaadja a járat útvonalát. A visszatérési érték formátuma a következő legyen: `[Induló kikötő] -> [Cél kikötő]`.

Készítsen még egy **MenetidoPercben** metódust, amely visszaadja a járat menetidejét percben. A számításhoz a `Menetido` adattag szöveges tartalmát kell feldolgozni. A lehetséges formátumok például:

- 8 perc
- 30 perc
- 1 óra
- 1 óra 15 perc
- 1 óra 50 perc

Készítsen még egy **Terheles** absztrakt metódust, amely visszaadja a járat terhelését. A kiszámításhoz az osztály leszármazottjai eltérő szabály szerint dolgozzanak!

Írja felül a **ToString** metódust, és az alábbi formátumban adja vissza a példány adatait. Ha nem tudja a `ToString`-gel megoldani, akkor a **Listaz** metódusban készítse el a kódrészletet a **Menetrend** osztályban. (Pontlevonással jár)

Minta:

`[Dátum] [Indulás]-[Érkezés] [Hajó neve], [Induló kikötő] -> [Cél kikötő], menetidő: [menetidő]`

Készítsen egy **MenetrendiJarat** osztályt, amely a fenti absztrakt osztályból származik. A menetrendi járat esetében a **Terheles** metódus az alábbi módon számoljon: *menetidő percben* * 1. Készítsen egy **SetaJarat** osztályt, amely szintén a fenti absztrakt osztályból származik. A sétajárat esetében a **Terheles** metódus az alábbi módon számoljon: *menetidő percben* * 2. Készítsen egy **KompJarat** osztályt, amely szintén a fenti absztrakt osztályból származik. A kompjárat esetében a **Terheles** metódus az alábbi módon számoljon: *menetidő percben* * 3.

A leszármazott osztályok valósítsák meg az **IComparable** interfészt, és az összehasonlítás alapja a **menetidő percben** legyen. Így a járatok rendezhetők legyenek menetidő szerint növekvő sorrendben. A `List<T>.Sort()` metódus a beépített rendezéshez az objektum saját összehasonlítását használja, ha a típus implementálja az **IComparable** interfészt.

Ezután hozzon létre egy **Menetrend** nevű osztályt, amely a fentebb létrehozott osztályoknak megfelelő járatokat egy **Listában** tárolja, amely bárhol elérhető legyen.

A nyilvános metódusok a következők legyenek:

- **Beolvas()**: a főprogramtól átvett fájl elérés alapján olvassa be a fájlt a tanult módon! Az első sort, amely fejléceket tartalmaz, hagyja ki! Minden sorból készítsen egy megfelelő típusú objektumot a típus mező alapján. Ha a típus értéke menetrendi, akkor **MenetrendiJarat** objektum jöjjön létre. Ha a típus értéke séta, akkor **SetaJarat** objektum jöjjön létre. Ha a típus értéke komp, akkor **KompJarat** objektum jöjjön létre. Ha a fájl nem található, kezelje az alábbi üzenettel: „**Hiba, a megadott fájl nem található!**”. Minden más hibát elegendő általánosan kezelnie! Az adatokat a listában tárolja!
- **Listaz()**: táblázatos formában írja ki a tárolt adatokat a mintának megfelelően.
- **AtlagMenetido()**: visszaadja a tárolt járatok átlagos menetidejét percben.
- **Leghosszabb()**: megkeresi azt a járatot, amelynek a legnagyobb a menetideje.
- **LegkisebbTerheles()**: megkeresi azt a járatot, amelynek a legkisebb a terhelése.
- **KeresHajoAlapjan()**: kilistázza azokat a járatokat, amelyek hajónevében szerepel a megadott részlet.
- **TípusSzerint()**: kilistázza az adott típushoz tartozó járatokat.
- **UjJaratHozzaadasa()**: bővíti a listát új elem hozzáadásával. Paraméterül egy **Jarat** objektumot vesz át.
- **RendezMenetidoSzerint()**: rendezi a listát menetidő szerint növekvő sorrendbe az **IComparable** megvalósítása alapján. A saját típusok rendezése C#-ban így közvetlenül elvégezhető a `lista.Sort()` metódusával.
- **Mentes()**: kiírja a teljes adatbázist egy új fájlba, kivételkezeléssel.

Ne használjon a feladat megoldásához olyan egyéb megoldási technikát, amelyet órai kereteken belül nem tanultunk! Ahol a függvény visszatérési értékkel rendelkezik, ott ne történjen kiíratás, csak a főfüggvényben! Csak a futó, és fordítási hiba nélküli programkódok értékelhetők!

Egészítse ki a Program.cs-t az alábbiak szerint:

Kérje be a fájl elérését a felhasználótól, példányosítsa a **Menetrend** osztályt, és olvassa be az adatokat! Készítsen végtelen ciklusban egy menüt a mintának megfelelően. Gondoskodjon róla, hogy gombnyomás után törlődjön a konzol tartalma, és jelenjen meg újra a menü!

1. Járatok listázása
2. Átlagos menetidő lekérdezése
3. Leghosszabb járat megjelenítése
4. Legkisebb terhelésű járat megjelenítése
5. Keresés hajónév alapján
6. Szűrés típus szerint
7. Új járat hozzáadása
8. Rendezés menetidő szerint
9. Mentés fájlba
10. Kilépés

Példák a menetidő értelmezéséhez

A MenetidoPercben() metódus működésére példák:

- 8 perc → 8
- 30 perc → 30
- 1 óra → 60
- 1 óra 15 perc → 75
- 1 óra 50 perc → 110

A típus mező alapján létrehozandó objektumpéldák:

- menetrendi → MenetrendiJarat
- séta → SetaJarat
- komp → KompJarat